

Machine Learning

Contents

1. Introduction	4
What Does It Mean to Learn?	4
Traditional Programming and Machine Learning	4
When Machine Learning Is Useful	4
The Three Main Learning Paradigms	4
Supervised Learning	5
Unsupervised Learning	5
Reinforcement Learning	5
Designing a Learning System	5
Typical Failure Modes	6
Takeaway	6
2. Mathematical Preliminaries / Machine Learning Workflow	6
The End-to-End Flow	6
Data Objects and Attributes	7
Data Quality	8
IQR Flagging	8
Three-Sigma Flagging	8
Sampling and Partitioning	8
Scaling and Encoding	9
Feature Engineering	9
Takeaway	9
Linear Models for Regression	9
From inputs to a prediction	10
Two ways to fit the model	10
Direct solution	10
Gradient descent	11
Measuring prediction quality	11
Linear does not mean straight-only	11
Bias, variance, and regularization	12
Practical checklist	12
Linear Models for Classification	12
Score and decision boundary	13
Discriminant functions	13
Generative and discriminative approaches	13

Decisions need thresholds and costs	14
A compact example	14
Practical checklist	14
Logistic Regression	14
Probability from a linear score	14
Why not use linear regression for probability?	15
Cross-entropy loss	15
Compact example	16
Regularization and preprocessing	16
Scope boundary	16
Practical checklist	16

1. Introduction

Machine learning matters when a useful rule is easier to learn from examples than to write by hand. A spam filter, for example, can learn which combinations of words and sender patterns predict spam instead of depending on a permanently growing list of human-written rules.

[Detailed](#) · [External Sources](#)

What Does It Mean to Learn?

The lecture used Tom Mitchell's task–performance–experience framing. A program learns when its performance improves on a task because of experience.

- Task, T: what the system must do.
- Performance measure, P: how success is measured.
- Experience, E: the data or interaction from which it learns.

For handwriting recognition:

- T is assigning the correct character to an image.
- P could be the percentage of unseen images classified correctly.
- E is a collection of images paired with human-provided labels.

This framing prevents a vague statement such as “build an intelligent system.” It forces us to identify an observable task, a measurable result, and the evidence available for learning.

Traditional Programming and Machine Learning

In traditional programming, a developer supplies data and explicit rules; the computer returns an output. In supervised machine learning, the developer supplies examples of inputs and desired outputs; a learning algorithm estimates a model that can later produce outputs for new inputs.

The learned model is not magic and does not “understand” in the human sense. It is a mathematical mapping whose parameters were adjusted to perform well on observed examples.

When Machine Learning Is Useful

The lectures emphasized several useful situations:

- The rule is hard to express but examples are available, as in handwriting or speech recognition.
- Patterns change over time, as in spam or fraud detection.
- The model must be personalized, as in recommendations.
- The volume or dimensionality of data makes manual rule writing impractical.
- Decisions improve through repeated interaction, as in game-playing or robotic control.

Machine learning is not automatically the right choice. A stable payroll calculation, for example, is clearer, cheaper, and safer as an explicit program.

The Three Main Learning Paradigms

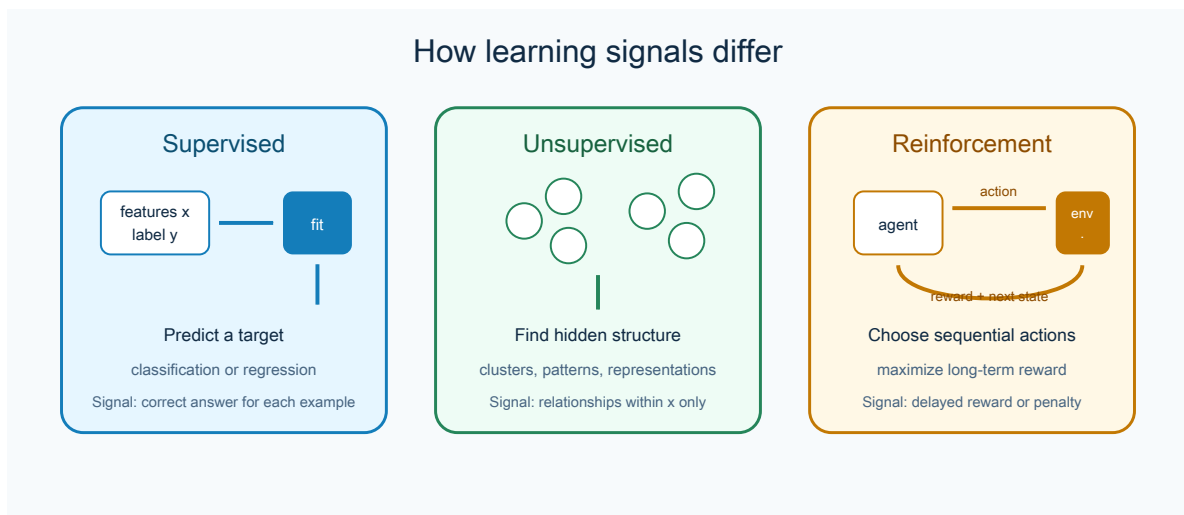


Figure 1. The kind of feedback available determines the learning paradigm.

Supervised Learning

Supervised learning uses labelled examples (x, y) . The input x contains features; the target y is the desired answer.

- Classification: y is a category, such as spam/not-spam or job-offered/not-offered.
- Regression: y is a numerical value, such as house price or humidity.

The goal is not merely to reproduce the training examples. It is to generalize to previously unseen cases.

Unsupervised Learning

Unsupervised learning receives inputs x without target labels. It searches for useful structure, such as customer groups or lower-dimensional representations. In clustering, a useful result usually places similar observations together and separates dissimilar groups.

Reinforcement Learning

An agent acts in an environment and receives rewards or penalties. The challenge is sequential: an action can affect later states and later rewards. The objective is therefore long-term reward, not simply the correctness of one isolated prediction.

The lecture also mentioned semi-supervised learning, where a small labelled set is combined with a larger unlabelled set.

Designing a Learning System

A practical learning system requires more than selecting an algorithm.

- 1 Define the task and the real-world decision it supports.
- 2 Choose a performance measure that reflects the cost of mistakes.
- 3 Decide what experience is available and whether it represents future cases.
- 4 Choose the target and features without leaking future information.
- 5 Select a model family and training procedure.
- 6 Evaluate on data that was not used to fit or tune the model.

7 Monitor the deployed system because data and behaviour can change.

The workflow and data-quality stages are developed in [Know More](#).

Typical Failure Modes

- Unrepresentative experience: training examples differ from deployment cases.
- Poor target definition: the label is convenient but does not measure the real goal.
- Data leakage: training uses information that would be unavailable at prediction time.
- Wrong metric: a high aggregate score hides expensive errors or weak performance on an important group.
- Overfitting: the model learns accidental details of the training sample.
- Feedback effects: model decisions change the data that will be collected later.

Takeaway

Machine learning is a disciplined way to improve performance from experience. Start with T, P, and E; then choose the learning paradigm from the feedback available. The quality of the task definition and evidence usually matters more than the novelty of the algorithm.

2. Mathematical Preliminaries / Machine Learning Workflow

The current lecture coverage for this handout topic is the Machine Learning Workflow. The mathematical-preliminaries branch was not substantively taught in these five lectures, so this note does not create an unrelated mathematics survey.

A machine-learning model is only one stage in a larger evidence pipeline. Reliable work starts by understanding the data, separating evaluation data early, and ensuring every learned preprocessing step is fitted without access to the final test set.

[Detailed](#) · [External Sources](#)

The End-to-End Flow

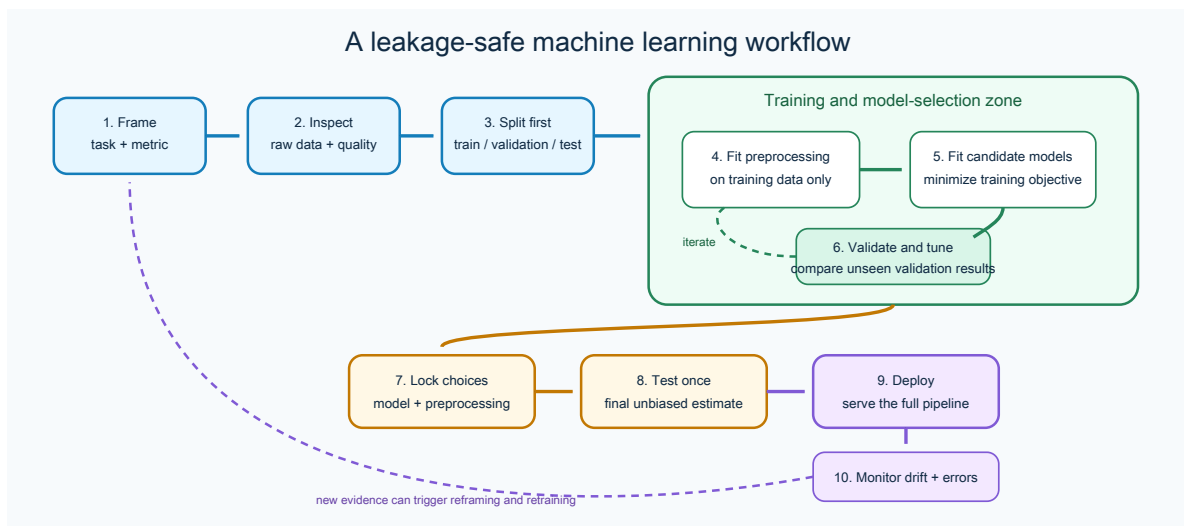


Figure 1. Split before fitting transformations; reserve the test set for the final estimate.

The lecture organized practice around data representation, parameter optimization, and model evaluation. A robust workflow expands that idea:

- 1 Frame the task, target, metric, and prediction time.
- 2 Collect and inspect relevant data.
- 3 Split into training, validation, and test roles.
- 4 Fit cleaning, imputation, encoding, scaling, and feature selection on training data.
- 5 Fit candidate models on transformed training data.
- 6 Use validation data or cross-validation to tune choices.
- 7 Lock the complete pipeline.
- 8 Evaluate once on the held-out test set.
- 9 Deploy the same transformation-and-model pipeline.
- 10 Monitor data quality, drift, errors, and downstream effects.

Data Objects and Attributes

An object is one record, instance, case, or sample. An attribute is a measured property, field, variable, or feature.

Attribute scale	What comparisons are meaningful?	Example
Nominal	equality or difference	city, colour, product code
Ordinal	equality and order	low/medium/high, rating level
Interval	order and differences	Celsius temperature, calendar year
Ratio	order, differences, and ratios	mass, duration, Kelvin temperature

Attributes may also be discrete (countable values) or continuous (values on a numeric continuum). These distinctions affect encoding, summaries, distance calculations, and valid interpretations.

Data Quality

The lectures emphasized that poor data can deny credit to qualified applicants or approve risky applicants. Common problems include:

- measurement noise;
- invalid or inconsistent values;
- missing values;
- duplicates created when sources are merged;
- rare observations or outliers;
- unrepresentative or imbalanced samples;
- misleading, irrelevant, or redundant features.

Cleaning is not equivalent to deleting every unusual row. First determine whether an unusual value is an error, a valid rare case, or the event the analysis is supposed to detect.

IQR Flagging

Let Q_1 and Q_3 be the first and third quartiles. The interquartile range is:

$$IQR = Q_3 - Q_1$$

Tukey's common inner fences are:

$$\begin{aligned}\text{lower fence} &= Q_1 - 1.5 \times IQR \\ \text{upper fence} &= Q_3 + 1.5 \times IQR\end{aligned}$$

Values beyond these fences are candidates for investigation.

Three-Sigma Flagging

For data adequately described by a normal distribution, approximately 99.73% of the population lies within $\text{mean} \pm 3 \times \text{sd}$, where mean is the population mean and sd the population standard deviation.

Correction: the lecture slide rounds this to 99%. The exact normal-distribution value is about 99.73%, and the rule should not be applied blindly to strongly skewed or multimodal data. [External Sources](#)

Sampling and Partitioning

A sample supports generalization only when it represents the cases on which the model will operate. Small samples create sampling noise; flawed collection can create sampling bias even when the sample is large.

- Simple random sampling gives eligible observations a defined random chance of selection.
- Stratified sampling preserves important group proportions, such as class labels.
- Cluster sampling samples groups rather than independent individuals.
- Under-sampling removes majority examples; over-sampling adds or repeats minority examples. Both alter the training distribution and must be applied inside the training process only.

The training set estimates model parameters. The validation set or cross-validation process selects models and hyperparameters. The test set estimates performance after choices are locked.

Scaling and Encoding

Min-max scaling to $[0, 1]$ is:

$$x' = (x - \min_{\text{train}}) / (\max_{\text{train}} - \min_{\text{train}})$$

Standardization is:

$$z = (x - \mu_{\text{train}}) / \sigma_{\text{train}}$$

The subscript `train` is essential: fit the minimum, maximum, mean, standard deviation, imputer, encoder, and feature selector on training data only. Apply those learned transformations unchanged to validation, test, and production data.

Validation note: both displayed formulas require a nonzero training range or standard deviation. A constant feature needs an explicit policy, such as removal or a documented library convention; it must not be divided by zero.

Validation note: standardization is not restricted to normally distributed data. It is useful when feature scale affects optimization, distance, or regularization; its sensitivity to outliers must still be considered.

Categorical variables may be represented with one-hot encoding or another explicit encoding. Label encoding can accidentally introduce a numerical order when categories are nominal.

Feature Engineering

- Extraction: derive a lower-dimensional representation.
- Selection: retain useful existing features and remove irrelevant or redundant ones.
- Construction: combine raw features using domain knowledge or interactions.
- Transformation: change representation, such as logarithms, scaling, or binning.

Correction: continuous features do not universally need discretization. Decision trees and k-nearest-neighbour methods can use continuous inputs directly, and Gaussian Naive Bayes models continuous features. Binning is a deliberate modelling choice with information-loss and boundary effects.

Takeaway

Split early, fit preprocessing only on training data, preserve the same pipeline for every later input, and treat unusual data as evidence to investigate. A trustworthy workflow protects the evaluation boundary from the first transformation onward.

Linear Models for Regression

Regression predicts a continuous quantity, such as price, demand, temperature, or travel time. A linear model is a useful starting point because its predictions are easy to compute and its parameters have a clear interpretation.

[Detailed](#) · [External Sources](#)

From inputs to a prediction

For one feature, simple linear regression uses

$$\hat{y} = w_0 + w_1 x$$

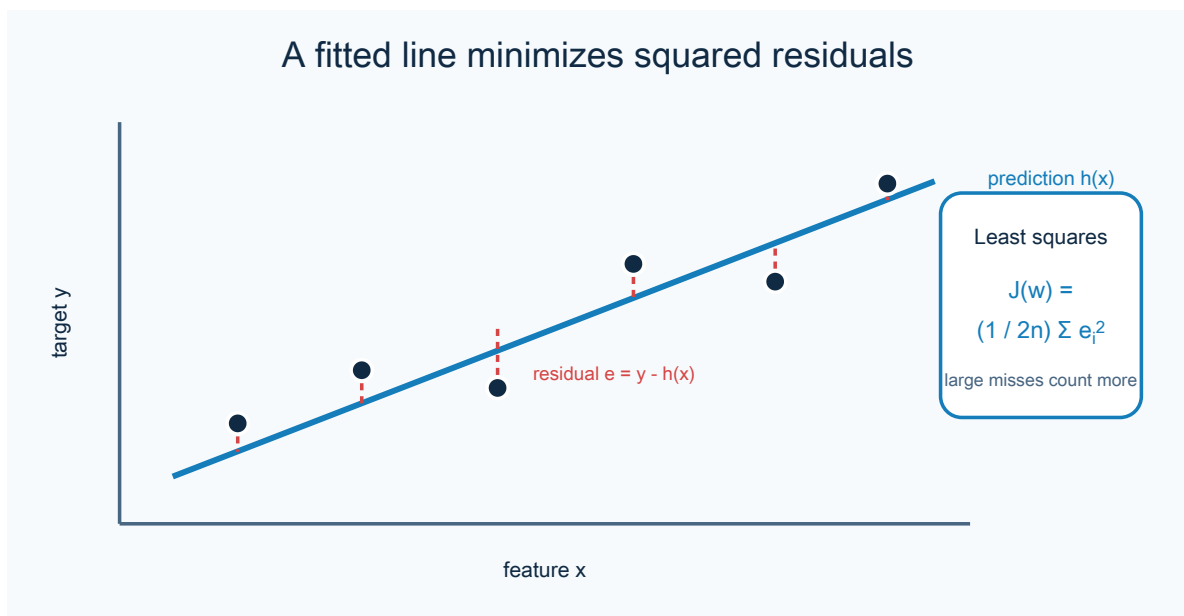
Here, x is the input, $\hat{y}$ is the prediction, w_0 is the intercept, and w_1 is the slope. With d features, the model becomes

$$\hat{y} = w_0 + \sum_{j=1}^d w_j x_j = \mathbf{x}_{\text{aug}}^T \mathbf{w}$$

where the augmented vector includes a leading 1 for the intercept:

$$\mathbf{x}_{\text{aug}} = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ \vdots \\ x_{(d)} \end{bmatrix}$$

For an entire dataset, the prediction vector is $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$.



A fitted line passing through a cloud of observations, with residuals shown as vertical gaps.

The residual for observation i is $e_i = \hat{y}_i - y_i$. Least-squares regression chooses parameters that minimize

$$J(\mathbf{w}) = \frac{1}{2n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

Squaring prevents positive and negative errors from cancelling and penalizes larger errors more strongly.

Two ways to fit the model

Direct solution

Setting the gradient of the least-squares objective to zero gives the normal equations:

$$\mathbf{X}^T \mathbf{X} \mathbf{w} = \mathbf{X}^T \mathbf{y}$$

If $\mathbf{X}^T \mathbf{X}$ is invertible, this gives

$$w = (X^T X)^{-1} X^T y$$

Correction. Explicit inversion is not always possible or numerically desirable. When columns are dependent or the system is ill-conditioned, use a least-squares solver, matrix factorization, or the Moore–Penrose pseudoinverse instead of assuming the inverse exists.

Gradient descent

Gradient descent starts with an initial parameter vector and repeatedly updates every parameter:

$$w_j \leftarrow w_j - \alpha (1/n) \sum_{i=1}^n (\hat{y}_i - y_i) x_{ij}$$

where $\alpha > 0$ is the learning rate and $x_{i0} = 1$. A very small learning rate makes slow progress; a very large one can overshoot or diverge. Feature scaling often makes one learning rate work more evenly across parameters.

- Batch gradient descent: uses all training examples per update.
- Stochastic gradient descent: uses one example per update.
- Mini-batch gradient descent: uses a small subset and is the usual practical compromise.

The direct method is attractive for modest problems. Iterative methods become useful when the dataset or feature set is large. [Know More](#)

Measuring prediction quality

Mean squared error (MSE):

$$MSE = (1/n) \sum_i (\hat{y}_i - y_i)^2$$

Lower is better, but its unit is squared.

Root mean squared error (RMSE):

$$RMSE = \sqrt{MSE}$$

RMSE is expressed in the target's original unit.

R-squared:

$$R^2 = 1 - [\sum_i (y_i - \hat{y}_i)^2 / \sum_i (y_i - \bar{y})^2]$$

R-squared compares the model with predicting the sample mean. It can be negative on evaluation data. The displayed ratio is undefined when every evaluated target equals the same value, so software may apply a documented finite-value convention.

Always compute evaluation metrics on data not used to fit the parameters.

Linear does not mean straight-only

A model can be nonlinear in the original input but linear in its parameters. For example,

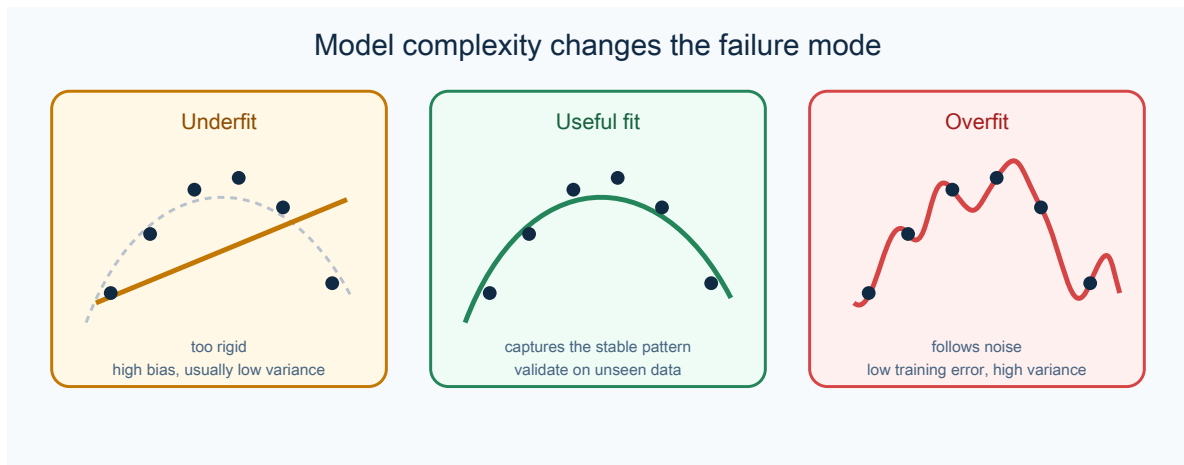
$$\hat{y} = w_0 + w_1 x + w_2 x^2 = w^T \phi(x)$$

with basis vector

$$\phi(x) = \begin{bmatrix} 1 \\ x \\ x^2 \end{bmatrix}$$

Increasing polynomial degree increases flexibility but can also increase sensitivity to noise.

Bias, variance, and regularization



Model complexity trades underfitting against overfitting, with validation error lowest between them.

- High bias: the model is too restrictive and underfits both training and evaluation data.
- High variance: the model follows training details too closely and performs poorly on new data.

Regularization discourages excessive parameter magnitude. Ridge uses an L2 penalty:

$$J(w) + \lambda \sum_{j=1}^d w_j^2$$

Lasso uses an L1 penalty:

$$J(w) + \lambda \sum_{j=1}^d |w_j|$$

Elastic net combines L1 and L2 penalties.

The intercept is commonly excluded from the penalty. Larger λ increases shrinkage and may increase bias while reducing variance. Select it using validation data rather than the test set.

Practical checklist

- 1 Separate training, validation, and test roles before fitting.
- 2 Inspect target units, missing values, and suspicious observations.
- 3 Scale features when using gradient-based fitting or regularization.
- 4 Compare training and validation errors to diagnose fit.
- 5 Report a metric whose units and limitations are clear.

Linear Models for Classification

Classification predicts a category rather than a continuous number. A linear classifier summarizes the input with a weighted score and separates classes using a line, plane, or higher-dimensional hyperplane.

[Detailed](#) · [External Sources](#)

Score and decision boundary

For a feature vector $\mathbf{x}$, define the score

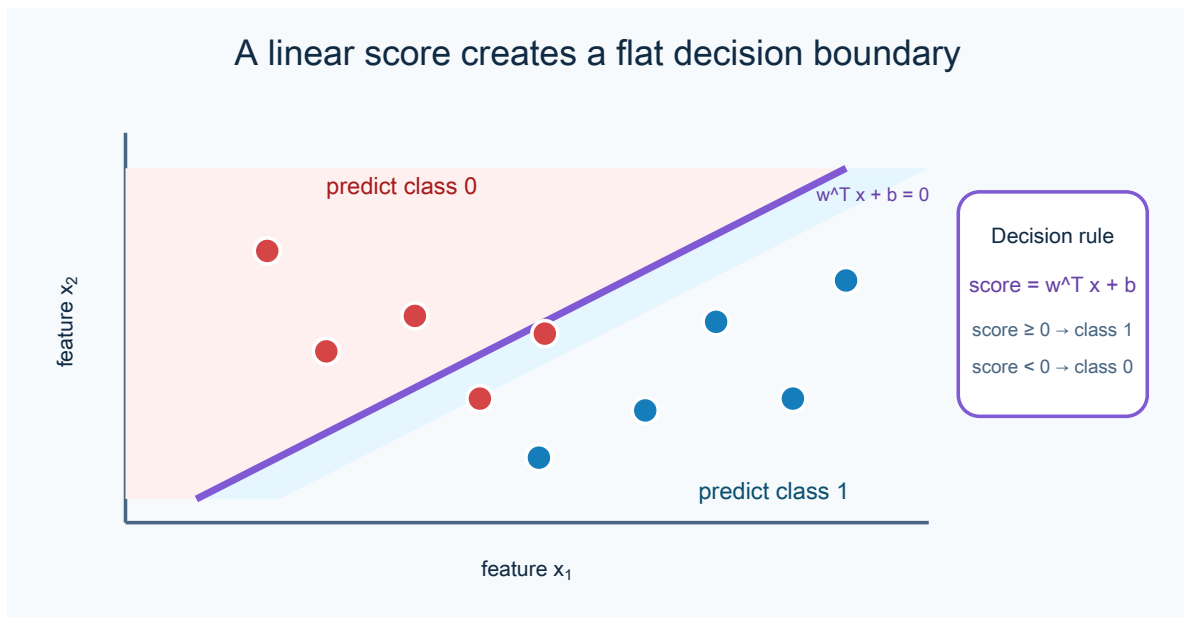
$$a(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$$

where $\mathbf{w}$ contains feature weights and b is the intercept. A basic binary rule is

$$\hat{y} = \begin{cases} 1 & \text{if } a(\mathbf{x}) \geq 0 \\ 0 & \text{if } a(\mathbf{x}) < 0 \end{cases}$$

The decision boundary is the set of inputs satisfying

$$\mathbf{w}^T \mathbf{x} + b = 0$$



A two-dimensional linear decision boundary separating two labeled classes, with a normal vector perpendicular to the boundary.

The vector $\mathbf{w}$ is perpendicular to the boundary. Its direction points toward increasing scores, while b shifts the boundary without rotating it.

Discriminant functions

A discriminant function assigns a score to each class. In multiclass notation, choose

$$\hat{y} = \arg \max_k g_k(\mathbf{x})$$

For a linear discriminant, $g_k(\mathbf{x}) = \mathbf{w}_k^T \mathbf{x} + b_k$. The boundary between classes j and k occurs where $g_j(\mathbf{x}) = g_k(\mathbf{x})$.

The lectures used this idea to connect geometric separation with probabilistic classifiers. Logistic regression, covered next, transforms a linear score into a binary class probability. [Know More](#)

Generative and discriminative approaches

- A generative classifier models how each class could generate inputs, commonly through $p(\mathbf{x} | y)$ and $p(y)$, then applies Bayes' rule.
- A discriminative classifier models $p(y | \mathbf{x})$ directly or learns a decision boundary without modeling the full input distribution.

Naive Bayes is a standard generative example; logistic regression is a discriminative example.

Validation note. It is tempting to say that generative models always win with little data and discriminative models always win with much data. That is not a universal rule. Performance depends on whether model assumptions fit the data, optimization, regularization, dimensionality, and the evaluation setting.

Decisions need thresholds and costs

A probability estimate does not determine a decision until a threshold or loss rule is chosen. With equal costs, a binary classifier often uses threshold 0.5. When a false negative is more costly than a false positive, a lower threshold may be appropriate.

Changing the threshold changes predicted labels but does not retrain the underlying probability model. Evaluate the resulting trade-off on validation data and reserve the test set for final assessment.

A compact example

Let $a(x) = 2x_1 - x_2 - 1$.

- For $x = (2, 1)$, $a = 2$, so the rule predicts class 1.
- For $x = (0, 2)$, $a = -3$, so the rule predicts class 0.
- Points satisfying $2x_1 - x_2 - 1 = 0$ lie on the boundary.

The score's sign controls the class. Correction: its magnitude is not itself a geometric distance because rescaling both w and b changes the score without moving the boundary. The perpendicular distance is

$$|w^T x + b| / \|w\|_2$$

A raw score is also not automatically a calibrated probability.

Practical checklist

- 1 Define the positive class and error costs before choosing a metric.
- 2 Fit preprocessing only on training data.
- 3 Inspect class balance and use more than accuracy when imbalance matters.
- 4 Tune thresholds on validation data, not on the test set.
- 5 Check whether a linear boundary is plausible for the available features.

Logistic Regression

Logistic regression is a binary classification model that turns a linear score into a probability. It keeps the interpretability of a linear boundary while ensuring predictions lie between zero and one.

[Detailed](#) · [External Sources](#)

Probability from a linear score

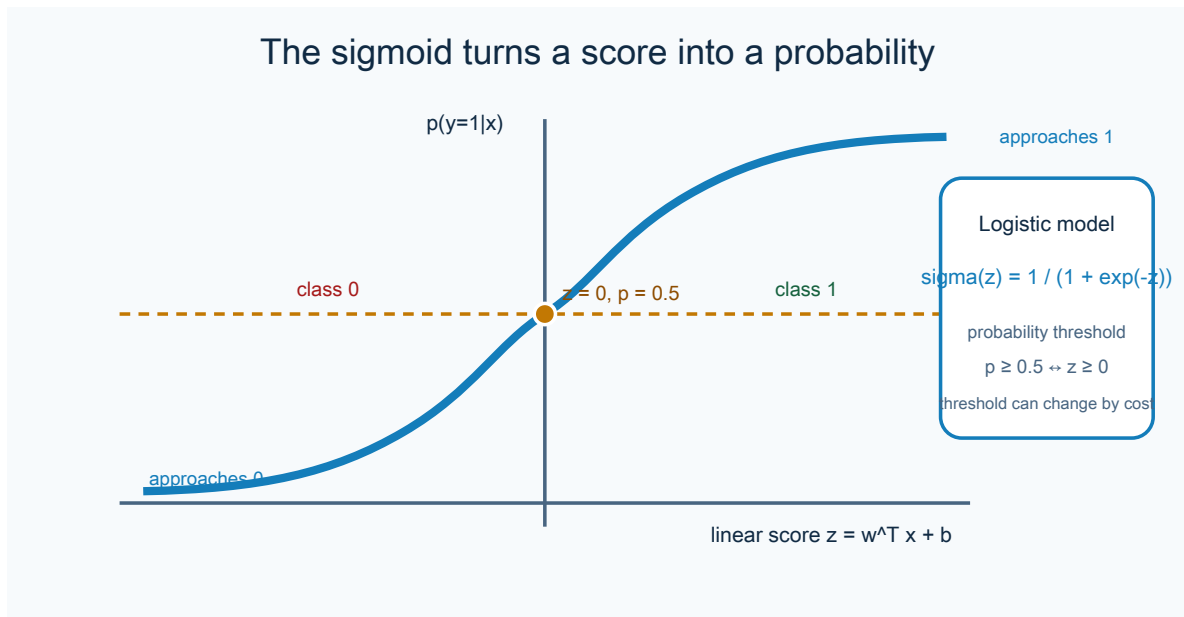
First compute

$$z = w^T x + b$$

Then apply the sigmoid function:

$$p = \sigma(z) = 1 / (1 + e^{-z})$$

The model interprets p as $P(y = 1 | x)$. As z increases, p approaches 1; as z decreases, p approaches 0.



The sigmoid curve maps every real-valued score to a probability between zero and one.

With a probability threshold of 0.5, predict class 1 when $p \geq 0.5$. Because $\text{sigmoid}(0) = 0.5$, the corresponding boundary is $w^T x + b = 0$.

Why not use linear regression for probability?

An unconstrained linear prediction can be below 0 or above 1. Clipping those values loses a coherent probability model and does not produce the logistic loss used for Bernoulli outcomes. Logistic regression instead models the log-odds linearly:

$$\ln(p / (1 - p)) = w^T x + b$$

A one-unit increase in feature x_j , with other features fixed, multiplies the odds by $\exp(w_j)$.

Cross-entropy loss

For binary label y in $\{0, 1\}$, the loss for one example is

$$L(y, p) = -[y \ln(p) + (1 - y) \ln(1 - p)]$$

The mean training objective is

$$J(w, b) = (1 / n) \sum_{i=1}^n L(y_i, p_i)$$

This loss strongly penalizes confident wrong predictions. For the unregularized binary model, its gradients are

$$\partial J / \partial w = (1 / n) X^T (p - y)$$

and

$$\partial J / \partial b = (1 / n) \sum_{i=1}^n (p_i - y_i)$$

Gradient descent applies $w \leftarrow w - \alpha \cdot dJ/dw$ and $b \leftarrow b - \alpha \cdot dJ/db$.

Compact example

Suppose normalized academic score and aptitude score are both 1, with $b = -1$, $w_1 = 0.8$, and $w_2 = 0.4$. Then

$$z = -1 + 0.8(1) + 0.4(1) = 0.2$$

and

$$p = \sigma(0.2) \approx 0.550$$

At threshold 0.5, the model predicts the positive class. The probability is only modestly above the threshold, so it should not be described as high confidence.

Regularization and preprocessing

Logistic regression often includes an L2, L1, or elastic-net penalty to control coefficient magnitude. Scale features when their units differ substantially, particularly when regularization or gradient-based solvers are used. Fit every preprocessing step on training data only.

Current-version note. Solver and penalty combinations vary by software implementation. Check the documentation for the installed version rather than assuming every solver supports every penalty.

Scope boundary

The 30 August lecture developed binary logistic regression. Multiclass logistic regression appears later in the handout sequence but was not substantively taught in the five supplied lectures, so it is not expanded here.

Practical checklist

- 1 Define which outcome is encoded as 1.
- 2 Inspect class balance and choose suitable metrics.
- 3 Scale and transform using training data only.
- 4 Tune regularization and the decision threshold on validation data.
- 5 Evaluate once on untouched test data and report the threshold used.